```
# Robustness of the gauge-invariant sign structure of T.  Pipeline per realisation:
invariance algebra (antisymmetric triples) ->
# Cartan (centralizer of a random element) -> weights per slot -> T in weight
eigenbasis -> support + integer-lattice invariants.
import numpy as np, itertools
from scipy.linalg import qr
TOL=1e-9
def cd_mul(x,y,v=0):
    n=len(x)
    if n==1: return x*y
    h=n//2; a,b=x[:h],x[h:]; c,d=y[:h],y[h:]
    cj=lambda z: np.concatenate([z[:1],-z[1:]]) if len(z)>1 else z
    if v==0: return np.concatenate([cd_mul(a,c,v)-cd_mul(cj(d),b,v),
cd_mul(d,a,v)+cd_mul(b,cj(c),v)])
    return np.concatenate([cd_mul(a,c,v)-cd_mul(d,cj(b),v),
cd_mul(cj(a),d,v)+cd_mul(c,b,v)])
def tform(v=0):
    E=np.eye(8); T=np.zeros((8,8,8))
    for i in range(8):
        for j in range(8): T[i,j,:]=cd_mul(E[i],E[j],v)
    return T
Sb=[]
for p in range(8):
    for q in range(p+1,8):
        M=np.zeros((8,8)); M[p,q]=1; M[q,p]=-1; Sb.append(M)
def trip(v,null=None):
    return [sum(v[s*28+k]*Sb[k] for k in range(28)) for s in range(3)]
def pipeline(T,seed):
    cols=[]
    for s in range(3):
        for M in Sb:
            r=[np.einsum('ai,ajk->ijk',M,T),np.einsum('bj,ibk-
>ijk',M,T),np.einsum('ck,ijc->ijk',M,T)][s]; cols.append(r.ravel())
    u,sv,vt=np.linalg.svd(np.array(cols).T); r=np.sum(sv>TOL); null=vt[r:];
d=null.shape[0]
    L=[trip(v) for v in null]
    cm=lambda X,Y:[X[s]@Y[s]-Y[s]@X[s] for s in range(3)]
    def co(t): return null@np.concatenate([np.array([M[p,q] for p in range(8) for q
in range(p+1,8)]) for M in t])
    ad=lambda X: np.array([co(cm(X,L[b])) for b in range(d)]).T
    rng=np.random.RandomState(seed); X=trip(rng.randn(d)@null)
    _,s2,v2=np.linalg.svd(ad(X)); cent=v2[np.sum(s2>1e-9):]; Hs=[trip(c@null) for c
in cent]
    adH=[ad(H) for H in Hs]; K=np.array([[np.trace(a@b) for b in adH] for a in adH]);
Kinv=np.linalg.inv(-K)
    co_=rng.randn(len(Hs)); V=[];F=[]
```

```
    for s in range(3):
        Hg=sum(c*H[s] for c,H in zip(co_,Hs)); w,VV=np.linalg.eig(Hg);
VV=VV/np.linalg.norm(VV,axis=0); V.append(VV)
        F.append(np.array([[ (v.conj()@H[s]@v/(v.conj()@v)).imag for H in Hs] for v
in VV.T]))
    nn=np.concatenate([np.einsum('ni,ij,nj->n',f,Kinv,f) for f in F]);
Ch=np.linalg.cholesky(Kinv/nn.mean())
    W=[f@Ch for f in F]
    Tw=np.einsum('abc,ai,bj,ck->ijk',T,V[0],V[1],V[2])
    return d,len(Hs),W,Tw,(sv[sv>TOL].min(),sv[sv<=TOL].max() if (sv<=TOL).any() else
0.0)
def int_left_kernel(Mat):
    m,n=Mat.shape; A=[list(map(int,Mat[i]))+[1 if j==i else 0 for j in range(m)] for
i in range(m)]; r=0
    for col in range(n):
        while True:
            rows=[i for i in range(r,m) if A[i][col]!=0]
            if not rows: break
            piv=min(rows,key=lambda i:abs(A[i][col])); A[r],A[piv]=A[piv],A[r];
done=True
            for i in range(r+1,m):
                if A[i][col]!=0:
                    q=A[i][col]//A[r][col]; A[i]=[a-q*b for a,b in zip(A[i],A[r])]
                    if A[i][col]!=0: done=False
            if done: r+=1; break
    ker=[A[i][n:] for i in range(r,m)]; assert all(all(x==0 for x in A[i][:n]) for i
in range(r,m))
    return np.array(ker),np.array([row[n:] for row in A],dtype=float),r
def support(Tw): return [tuple(x) for x in np.argwhere(np.abs(Tw)>TOL)]
def incidence(idx):
    M=np.zeros((len(idx),24),int)
    for r,(i,j,k) in enumerate(idx): M[r,i]=1; M[r,8+j]=1; M[r,16+k]=1
    return M
def invariants(arr,idx,C):
    return np.array([np.prod([arr[e]**int(ce) for e,ce in zip(idx,c) if ce!=0]) for c
in C])
# ---------- base realisation + completeness check
T0=tform(0); d,rk,W0,Tw0,gap=pipeline(T0,0); idx0=support(Tw0); M0=incidence(idx0)
print("base: algebra dim",d,"Cartan rank",rk,"| support entries",len(idx0),"| |entry|
range",np.round([abs(Tw0[e]) for e in idx0],6).min(),np.round([abs(Tw0[e]) for e in
idx0],6).max(),"| largest off-support |T'|
%.1e"%np.abs(np.where(np.abs(Tw0)>TOL,0,Tw0)).max(),"| algebra sv gap: kept>=%.2f
dropped<=%.1e"%gap)
C0,A0,r0=int_left_kernel(M0); print("saturated integer relation lattice:
rank",len(C0),"(= 32 - rank of incidence",np.linalg.matrix_rank(M0),")")
f0=invariants(Tw0,idx0,C0); print("character on saturated
basis:",np.round(f0.real).astype(int).tolist()," max deviation from +-1:
%.1e"%np.abs(f0-np.round(f0.real)).max())
import sympy as sp
Cold=[]
```

```
for v in sp.Matrix(M0.T.tolist()).nullspace():
    den=sp.ilcm(*[x.q for x in v]); Cold.append([int(x*den) for x in v])
Cold=np.array(Cold); Cm=Cold@np.linalg.pinv(C0.astype(float)); print("index of my
earlier (rational-nullspace) basis inside the saturated lattice: |det|
=",round(abs(np.linalg.det(Cm)),6))
# ---------- 1. random rephasing control (U(1) and general complex scalings)
rng=np.random.RandomState(11); u=[np.exp(2j*np.pi*rng.rand(24)),
rng.randn(24)+1j*rng.randn(24)]
for name,uu in zip(["random U(1) phases","random complex scalings"],u):
    arr=Tw0.copy()
    for (i,j,k) in idx0: arr[i,j,k]*=uu[i]*uu[8+j]*uu[16+k]
    print("rephasing control (%s): max change of the 14 invariants %.1e"%
(name,np.abs(invariants(arr,idx0,C0)-f0).max()))
# ---------- alignment machinery
def aligns(W1,W2):
    out=[]
    b=[]
    for w in W1[0]:
        if np.linalg.matrix_rank(np.array(b+[w]),tol=1e-6)>len(b): b.append(w)
        if len(b)==4: break
    b=np.array(b); G=b@b.T
    for tau in itertools.permutations(range(3)):
        def rec(img):
            k=len(img)
            if k==4:
                O=np.array(img).T@np.linalg.inv(b.T)
                if np.abs(O@O.T-np.eye(4)).max()<1e-6 and
all(all(np.min(np.linalg.norm(W2[tau[m]]-(O@w),axis=1))<1e-5 for w in W1[m]) for m in
range(3)): out.append((tau,O))
                return
            for w in W2[tau[0]]:
                if abs(w@w-G[k,k])<1e-5 and all(abs(w@img[j]-G[k,j])<1e-5 for j in
range(k)): rec(img+[w])
        rec([])
    return out
def widx(W,t,v):
    dd=np.linalg.norm(W[t]-v,axis=1); j=int(np.argmin(dd)); return j if dd[j]<1e-5
else None
def compare(W1,Tw1,idx1,W2,Tw2,idx2,C1):
    pos2={e:r for r,e in enumerate(idx2)}; res={"match":0,"differ":0,"support":0};
first=None
    for tau,O in aligns(W1,W2):
        pi=[]; ok=True
        for e in idx1:
            new=[None]*3
            for m in range(3): new[tau[m]]=widx(W2,tau[m],O@W1[m][e[m]])
            new=tuple(new)
            if None in new or new not in pos2: ok=False; break
            pi.append(new)
        if not ok or len(set(pi))!=len(idx1) or len(idx2)!=len(idx1):
```

```
res["support"]+=1; continue
        arr2=np.array([Tw2[p] for p in pi]); arr1=np.array([Tw1[e] for e in idx1])
        f1=np.array([np.prod([arr1[r]**int(c[r]) for r in range(len(idx1)) if
c[r]!=0]) for c in C1])
        f2=np.array([np.prod([arr2[r]**int(c[r]) for r in range(len(idx1)) if
c[r]!=0]) for c in C1])
        if np.allclose(f1,f2,atol=1e-7):
            res["match"]+=1
            if first is None: first=(arr1,arr2)
        else: res["differ"]+=1
    return res,first
def rephase_residual(arr1,arr2,M):
    # exact solve of  M*theta = phi (mod 2pi) using the unimodular row reduction A0*M
= [E;0]  (E has full row rank r0)
    rr=arr2/arr1; phi=np.angle(rr); E=(A0@M)[:r0]; psi=A0@phi
    th=np.linalg.lstsq(E,psi[:r0],rcond=None)[0]
    return np.abs(np.exp(1j*(M@th))-rr).max(), np.abs(np.abs(rr)-1).max()
# ---------- 2/3. other realisations
rs=np.random.RandomState(5)
def sp_():
    P=np.zeros((8,8)); perm=rs.permutation(8)
    for i in range(8): P[perm[i],i]=rs.choice([-1,1])
    return P
cases=[("different Cartan (seed 7)",T0,7),("different Cartan (seed 21)",T0,21)]
Pa=sp_(); cases.append(("relabelled basis: same signed permutation on all three
slots",np.einsum('abc,ai,bj,ck->ijk',T0,Pa,Pa,Pa),3))
P1,P2,P3=sp_(),sp_(),sp_(); cases.append(("independent signed permutations per
slot",np.einsum('abc,ai,bj,ck->ijk',T0,P1,P2,P3),4))
cases.append(("opposite algebra  (T[j,i,k])",np.transpose(T0,(1,0,2)),5))
T1=tform(1); x=np.random.RandomState(2).randn(2,8);
ok1=np.linalg.norm(cd_mul(x[0],x[1],1))-np.linalg.norm(x[0])*np.linalg.norm(x[1])
print("\nalternative Cayley-Dickson convention: |xy|=|x||y| deviation %.1e"%abs(ok1))
if abs(ok1)<1e-9: cases.append(("alternative Cayley-Dickson convention",T1,6))
for name,Tc,seed in cases:
    d2,rk2,W2,Tw2,g2=pipeline(Tc,seed); idx2=support(Tw2)
    res,first=compare(W0,Tw0,idx0,W2,Tw2,idx2,C0)
    line=f"{name}: algebra dim {d2}, rank {rk2}, support {len(idx2)} | alignments
tested {sum(res.values())}: invariants match {res['match']}, differ {res['differ']},
support mismatch {res['support']}"
    if first is not None:
        rr,mr=rephase_residual(first[0],first[1],M0); line+=f" | explicit rephasing
residual {rr:.1e}, modulus mismatch {mr:.1e}"
    print(line)
```